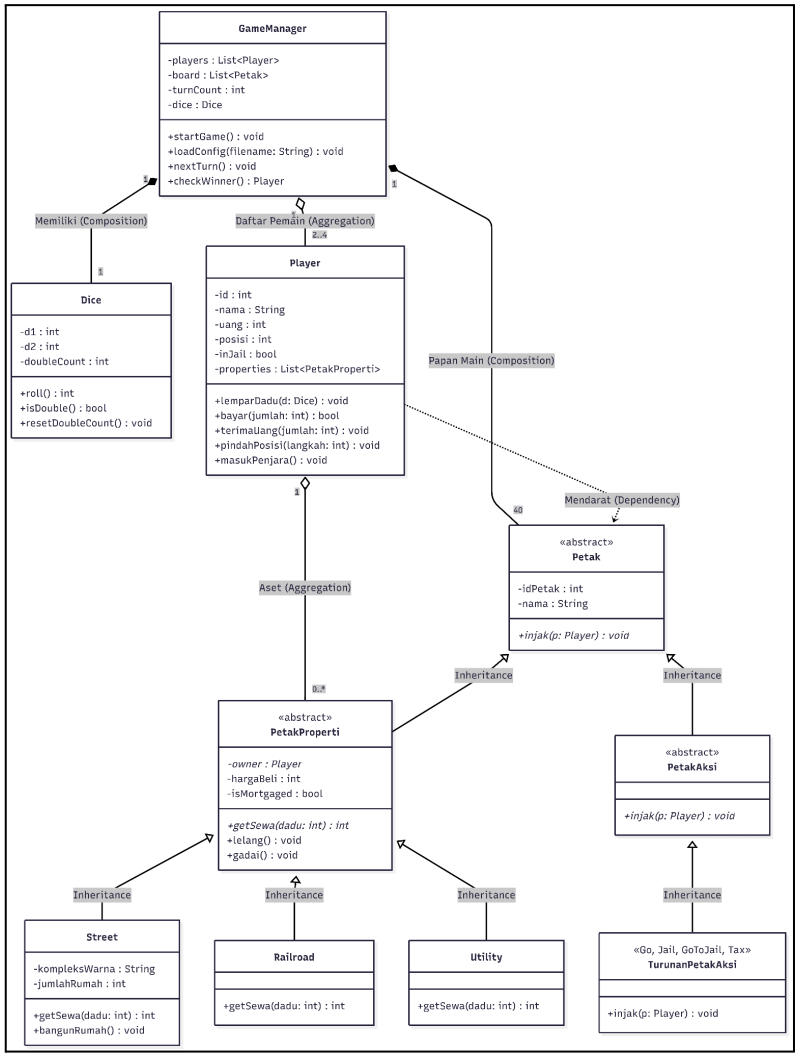
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : PSI
Nama Kelompok : PengenSleep

1. 13524113 / Fauzan Mohammad Abdul Ghani
2. 13524127 / Fazri Arrashyi Putra
3. 13524132 / Dika Pramudya Nugraha
4. 13524143 / Daniel Putra Rywandi S.
5. 13524145 / Dzakwan Muhammad Khairan Putra Purnama

PSI - PengenSleep

1. Diagram Kelas



Gambar 1.1 Diagram Kelas kelompok PengenSleep.

Berdasarkan diagram kelas di atas, arsitektur permainan Nimonspoli dibangun menggunakan beberapa jenis relasi objek untuk memastikan simulasi berjalan sesuai dengan logika dunia nyata. Berikut adalah rincian relasinya:

```
classDiagram
    class GameManager {
        -players : List~Player~
        -board : List~Petak~
        -turnCount : int
        -dice : Dice
        +startGame() void
        +loadConfig(filename: String) void
        +nextTurn() void
        +checkWinner() Player
    }

    class Dice {
        -d1 : int
        -d2 : int
        -doubleCount : int
        +roll() int
        +isDouble() bool
        +resetDoubleCount() void
    }

    class Player {
        -id : int
        -nama : String
        -uang : int
        -posisi : int
        -inJail : bool
        -properties : List~PetakProperti~
        +lemparDadu(d: Dice) void
        +bayar(jumlah: int) bool
        +terimaUang(jumlah: int) void
        +pindahPosisi(langkah: int) void
        +masukPenjara() void
    }
```

```
class Petak {
    <<abstract>>
    -idPetak : int
    -nama : String
    +injak(p: Player)* void
}

class PetakProperti {
    <<abstract>>
    -owner : Player*
    -hargaBeli : int
    -isMortgaged : bool
    +getSewa(dadu: int)* int
    +lelang() void
    +gadai() void
}

class Street {
    -kompleksWarna : String
    -jumlahRumah : int
    +getSewa(dadu: int) int
    +bangunRumah() void
}

class Railroad {
    +getSewa(dadu: int) int
}

class Utility {
    +getSewa(dadu: int) int
}

class PetakAksi {
    <<abstract>>
    +injak(p: Player)* void
}

class TurunanPetakAksi {
```

```
        <<Go, Jail, GoToJail, Tax>>
        +injak(p: Player) void
    }

    %% Relasi (Relationships)
    GameManager "1" *-- "1" Dice : Memiliki (Composition)
    GameManager "1" *-- "40" Petak : Papan Main (Composition)
    GameManager "1" o-- "2..4" Player : Daftar Pemain (Aggregation)
    Player "1" o-- "0..*" PetakProperti : Aset (Aggregation)

    Petak <|-- PetakProperti : Inheritance
    Petak <|-- PetakAksi : Inheritance

    PetakProperti <|-- Street : Inheritance
    PetakProperti <|-- Railroad : Inheritance
    PetakProperti <|-- Utility : Inheritance

    PetakAksi <|-- TurunanPetakAksi : Inheritance

    Player ..> Petak : Mendarat (Dependency)
```

1. Inheritance (Pewarisan): digunakan untuk mengelompokkan objek-objek yang memiliki sifat dasar sama, namun memiliki fungsi spesifik yang berbeda.
    - **Petak sebagai Kelas Induk (Parent Class):** Memiliki atribut dasar seperti ID petak dan nama.
    - **Turunan PetakProperti:** Mewakili petak yang bisa diperjualbelikan. Kelas ini diturunkan lagi menjadi tiga kelas yang lebih spesifik:
        - Street: Lahan biasa yang harganya bergantung pada level bangunan (jumlah rumah/hotel).
        - Railroad: Stasiun kereta, di mana harga sewanya bergantung pada jumlah stasiun lain yang dimiliki oleh pemilik yang sama.
        - Utility: Perusahaan Listrik/Air (PLN & PAM), di mana harga sewanya dihitung berdasarkan hasil lemparan dadu dikali faktor pengali.
    - **Turunan PetakAksi:** Mewakili petak yang memicu kejadian (*event*), seperti Jail (Penjara), GoToJail, Tax (Pajak), dan FreeParking.

2. Aggregation (Agregasi): adalah relasi "memiliki" (*has-a*) di mana objek yang dimiliki tetap bisa eksis meskipun pemiliknya tidak ada.

- **Player dan PetakProperti:** Seorang Player memiliki daftar (List/Vector) PetakProperti. Jika seorang pemain bangkrut dan dihapus dari permainan, PetakProperti tersebut tidak ikut hancur. Kepemilikannya hanya di-reset atau dipindahtangankan ke pemain lain (atau bank) melalui mekanisme lelang/kebangkrutan.

3. Composition (Komposisi): adalah relasi "memiliki" yang lebih kuat. Jika objek utamanya hancur, objek bagiannya ikut hancur karena keberadaannya sangat bergantung pada objek utama.
   - **GameManager dan Board (Papan):** Sistem utama (GameManager) merangkai 40 petak menjadi satu kesatuan papan permainan. Jika sesi permainan Nimonspoli selesai atau ditutup, maka papannya juga ikut direnggut dari memori.
   - **GameManager dan Dice:** Objek dadu adalah komponen absolut dari mesin permainan.

4. Association (Asosiasi): menunjukkan interaksi atau hubungan struktural antara dua kelas yang berdiri sendiri.
   - **Player dan Petak (Posisi Saat Ini):** Ada asosiasi yang mencatat di petak mana pemain sedang berdiri. Pemain mengetahui posisinya, dan petak bisa menerima interaksi dari pemain yang mendarat di atasnya.
   - **GameManager dan Player:** GameManager memantau dan menyimpan daftar seluruh pemain yang aktif untuk mengatur urutan giliran bermain (*turn-based system*).

5. Dependency (Kebergantungan): Relasi ini terjadi ketika sebuah kelas membutuhkan kelas lain untuk menjalankan metode tertentu (biasanya lewat parameter).
   - **Petak bergantung pada Player:** Ketika metode injak(Player& p) dipanggil pada suatu petak, petak tersebut membutuhkan data/referensi dari pemain yang menginjaknya untuk memotong uang (bayar sewa/pajak) atau memindahkan posisinya (masuk penjara).

***GameManager bergantung pada Sistem File (I/O):** Saat inisialisasi awal (*New Game*), GameManager bergantung pada file teks (.txt) eksternal untuk membaca nama kota, harga tanah, dan aturan konfigurasi lainnya.

2. Deskripsi Kelas

Pada bagian ini, dijelaskan rincian tanggung jawab setiap kelas dalam sistem Nimonspoli, serta peran atribut dan metodenya dalam mendukung jalannya permainan.

2.1. Kelas *GameManager*

- Tanggung Jawab: Menjadi otak utama (controller) yang mengatur siklus hidup permainan (game loop), mulai dari inisialisasi hingga menentukan pemenang.
- Penjelasan: Atribut players dan board menyimpan seluruh entitas permainan, sementara turnCount membatasi durasi gim. Method loadConfig() sangat krusial karena memungkinkan GameManager bersifat dinamis (tidak hardcoded) dengan membaca data dari file eksternal. Relasi Composition dengan Board dan Dice memastikan bahwa sistem pengatur inilah yang memegang kendali penuh atas komponen fisik permainan.

2.2. Kelas *Dice (Dadu)*

- Tanggung Jawab: Mengelola logika pengacakan angka dan mendeteksi kondisi khusus terkait lemparan dadu.
- Penjelasan: Atribut d1 dan d2 menyimpan hasil lemparan, sedangkan doubleCount digunakan untuk melacak berapa kali pemain mendapat angka kembar. Method isDouble() membantu GameManager memutuskan apakah pemain berhak jalan lagi atau justru harus masuk penjara jika melakukan pelanggaran kecepatan (3x double).

2.3. Kelas *Player*

- Tanggung Jawab: Merepresentasikan entitas pemain dan mengelola status ekonomi serta posisi mereka di papan.
- Penjelasan: Atribut uang, posisi, dan inJail adalah data privat yang hanya bisa diubah melalui interaksi resmi. Method bayar() dan terimaUang() menjadi pusat transaksi keuangan. Relasi Aggregation dengan PetakProperti memungkinkan pemain memiliki daftar aset yang tetap eksis meskipun pemain tersebut keluar dari permainan (bankrut).

2.4. Kelas *Petak (Abstract Class)*

- Tanggung Jawab: Menjadi kelas induk yang mendefinisikan kontrak dasar bagi semua kotak yang ada di papan.

- Penjelasan: Memiliki atribut idPetak dan nama. Method virtual murni injak(Player& p) memaksa setiap jenis petak memiliki perilaku unik saat didatangi pemain. Konsep Abstraksi disini memastikan GameManager bisa berinteraksi dengan 40 petak tanpa perlu tahu tipe spesifiknya secara langsung.

2.5. Kelas *PetakProperti (Abstract Class)*

- Tanggung Jawab: Mengelola logika kepemilikan dan status komersial dari sebuah petak.
- Penjelasan: Menambahkan atribut owner (pointer ke Player) dan isMortgaged. Method lelang() dan gadai() memberikan fleksibilitas ekonomi bagi pemain. Sebagai kelas abstrak, ia menyediakan kerangka bagi properti yang memiliki sistem sewa berbeda-beda melalui fungsi getSewa().

2.6. Kelas *Street (Lahan)*

- Tanggung Jawab: Mengelola properti jenis lahan yang dapat dikembangkan dengan bangunan.
- Penjelasan: Memiliki atribut spesifik kompleksWarna dan jumlahRumah. Method getSewa() di-override untuk menghitung biaya berdasarkan level bangunan. Kelas ini merupakan elemen utama dalam strategi monopoli warna untuk meningkatkan pendapatan pemain.

2.7. Kelas *Railroad (Stasiun) & Utility (PLN/PAM)*

- Tanggung Jawab: Mengelola properti khusus dengan skema perhitungan sewa yang unik.
- Penjelasan: Pada Railroad, sewa dihitung berdasarkan kuantitas stasiun yang dimiliki owner. Pada Utility, sewa dihitung secara dinamis menggunakan parameter hasil lemparan dadu. Keduanya melakukan override pada method getSewa() untuk mengimplementasikan aturan spesifik tersebut.

2.8. Kelas *PetakAksi & TurunanPetakAksi*

- Tanggung Jawab: Menangani petak-petak non-properti yang memicu kejadian atau prosedur tertentu.
- Penjelasan: Kelas ini tidak memiliki pemilik. Turunannya seperti Go (gaji), Jail (penjara), dan Tax (pajak) mengimplementasikan method injak() untuk langsung memanipulasi atribut Player (seperti menambah/mengurangi uang atau mengubah status inJail).

3. Justifikasi Desain

Pada desain yang sudah kami buat ada beberapa yang masih bisa diperbaiki, antara lain:

1. **Kelas Struktur Kartu:** Kelas ini bertanggung jawab untuk perilaku objek dari Kartu Kesempatan dan Dana Umum, kelas bisa saja terdiri dari Generic Class dan inheritance nya misalkan inheritance class MoveCard.
2. **Redundansi pada beberapa kelas dan method**: Kelas `TurunanAksi` harusnya bisa di simplifikasi dan cukup pada petak aksi dengan menambahkan beberapa state dan atribut tambahan.
3. **Perubahan nama atribut:** Pada class player harusnya `inJail: bool` bisa diganti dengan `status` agar bisa menampung status di penjara dan status bangkrut.

Selain dari ketiga Justifikasi di atas masih terbanyak redundansi pada method yang sama di beberapa kelas dan di kelas tersebut tidak terdapat atribut tambahan dari base class.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Di dalam proyek ini, konsep Inheritance digunakan untuk memodelkan hubungan antar berbagai jenis petak di atas papan permainan. Hal ini bertujuan untuk menghindari duplikasi kode dan menciptakan struktur data yang terorganisir.

Cuplikan kode pada file lagi include/models/Petak.hpp

```
class Petak {
protected:
  int id;
  std::string name;
  std::string shortName;
  std::string colorCode;
```

Kelas Abstrak Petak Berperan sebagai fondasi utama. Kelas ini mendefinisikan atribut protected yang pasti dimiliki oleh semua petak di papan dan interaksi dasar ketika pemain menginjak petak tersebut.

```
public:
  PetakProperti(int id, const std::string &name, int hargaBeli);
  virtual ~PetakProperti() = default;

  Player *getOwner() const;
  void setOwner(Player *newOwner);

  int getHargaBeli() const;

  bool getIsMortgaged() const;
  void setMortgaged(bool status);

  virtual void gadai();
  virtual void tebus();
  virtual void lelang();

  virtual int getSewa(int diceRoll = 0) const = 0;

  virtual std::string getGroupName() const = 0;
  virtual int getTotalWorth() const = 0;
  virtual void printDeed(IGameUI &ui) const = 0;

  void injak(Player &p, IGameUI &ui, int diceRoll = 0, std::vector<std::unique_ptr<Petak>>* board = nullptr,
std::vector<Player>* players = nullptr) override;
};
```

PetakProperti dan PetakAksi Mewarisi sifat dasar dari Petak. PetakProperti menambahkan atribut khusus yang berkaitan dengan kepemilikan, sedangkan PetakAksi mendefinisikan atribut yang berkaitan dengan pemicu sistem game non properti.

4.2. Method/Operator Overloading

Di dalam proyek ini, konsep *Method Overloading* diimplementasikan dengan memanfaatkan fitur *Default Arguments* (*Parameter Default*) bawaan C++. Fitur ini pada dasarnya adalah cara C++ mengimplementasikan *Method Overloading* tanpa harus menulis deklarasi fungsi yang sama berulang kali.

Cuplikan kode pada file include/models/Player.hpp:

```
class Player {
public:
  // Konstruktor Player dengan parameter default 'startingMoney = 1500'
  Player(int id, const std::string &name, int startingMoney = 1500);
};
```

Cuplikan kode pada file include/models/Card.hpp:

```
class SpecialCard {
public:
  // Konstruktor SpecialCard dengan parameter default 'value = 0' dan 'duration = 0'
  SpecialCard(SpecialCardType type, int value = 0, int duration = 0);
};
```

4.2.1 Penjelasan & Alasan Penggunaan

Penggunaan parameter default pada konstruktor *Player* dan *SpecialCard* merupakan *best-practice* dalam bahasa C++ untuk mensimulasikan *Method Overloading*. Keuntungan utama dari pendekatan ini sesuai dengan kasus tersebut adalah:

- *Pemanggilan Menjadi Fleksibel dan Intuitif*
  Objek *Player* dapat diinstansiasi dengan 2 parameter (misal: Player(1, "A")) jika ingin membuat player dengan uang awal standar M1500 (game default), atau bisa juga dipanggil dengan 3 parameter (misal: Player(1, "A", 5000)) jika di masa depan ada mode permainan khusus yang membutuhkan uang awal kustom. Ini membuktikan fleksibilitas dari Method Overloading.
- *Mengurangi Redundansi Kode (Clean Code)*
  Daripada menulis ulang deklarasi konstruktor secara terpisah (misalnya mendeklarasikan Player(int, string) lalu mendeklarasikan lagi Player(int, string, int)), kita cukup menyatakannya dalam satu baris deklarasi yang ringkas. Hal ini sangat menguntungkan karena mencegah kode menjadi kotor dan menyederhanakan proses pemeliharaan (maintainability).

4.3. Template & Generic Classes

Pada pengembangan tugas besar ini, konsep template dan generic classes tidak diimplementasikan dalam sistem yang dibangun. Keputusan ini diambil dengan mempertimbangkan kebutuhan aplikasi yang masih dapat dipenuhi secara optimal tanpa

memanfaatkan fitur generik tersebut, sehingga struktur kode tetap lebih sederhana dan mudah dipahami. Dengan demikian, seluruh komponen dirancang menggunakan pendekatan yang lebih spesifik sesuai kebutuhan fungsional sistem tanpa mengurangi kualitas maupun keterbacaan kode.

4.4. Exception

Tugas ini memakai beberapa exception handling di beberapa tempat, misalnya pada:

1. SaveManager.cpp

```
bool SaveManager::loadGame(GameManager &gm, const std::string &filename) {
  // ...
  try {
    int turnCount = 0, maxTurn = 0;
    if (!(in >> turnCount >> maxTurn))
      throw std::runtime_error("Format tidak valid (baris 1)."); // Lempar exception manual

    // ... proses loading lainnya ...

    int val = (valStr != "-") ? std::stoi(valStr) : 0; // Potensi exception dari std::stoi

  } catch (const std::exception &e) {
    // Menangkap semua kesalahan (baik throw manual maupun error sistem)
    std::cout << "Gagal memuat file! " << e.what() << "\n";
    return false;
  }
  return true;
}
```

Bagian ini menggunakan blok try-catch secara ekstensif untuk memastikan bahwa jika file simpanan (save file) rusak atau memiliki format yang salah, program bisa memberikan peringatan alih-alih langsung menutup diri.

- **Concept**: Digunakan untuk validasi integritas file eksternal.
- **Alasan**: Memuat file adalah operasi yang "berbahaya" karena isi file di luar kendali program. Jika ada satu baris yang salah format, fungsi std::stoi atau operator >> bisa gagal.

- **Keuntungan**: Program menjadi lebih aman. Tidak perlu mengecek setiap baris dengan if-else yang rumit, cukup kumpulkan semua potensi kegagalan di dalam blok try dan tangani hasilnya di catch.

2. TurnController.cpp

```
while (true) {
  std::string inp = ui.promptInput("\nPilih nomor kartu yang ingin dibuang: ");
  int choice = 0;
  try {
      choice = std::stoi(inp); // Mengonversi string ke integer
  } catch (...) {
      // Menangkap exception jika input bukan angka (misal: "abc")
  }

  if (choice < 1 || choice > sz) {
    ui.showMessage("[!] Nomor tidak valid.");
    continue;
  }
  // ...
}
```

- **Concept**: Menangani runtime error dari fungsi standar C++ (std::stoi).
- **Alasan**: Pengguna sering kali tidak sengaja memasukkan huruf saat diminta memasukkan angka. Fungsi std::stoi akan melempar std::invalid_argument jika inputnya bukan angka.
- **Keuntungan**: Mencegah program force close saat terjadi kesalahan input manusia. Program tetap berjalan dan hanya meminta ulang input yang benar.

4.5. C++ Standard Template Library

Proyek ini menggunakan template classes dari C++ Standard Library (STL) secara ekstensif, dan penggunaan STL untuk proyek ini sangat beragam. Berikut adalah cuplikan kode pada src/core/GameManager.cpp:

```
#include <vector>
class GameManager {
```

```
private:
  std::vector<Player>                        players;
  std::vector<std::unique_ptr<Petak>>        board;
  std::vector<std::unique_ptr<SpecialCard>>  specialDeck;
  std::vector<std::unique_ptr<SpecialCard>>  specialDiscard;
  std::vector<std::string>                   transactionLog;
  ...
};
```

4.5.1. Penjelasan & Alasan Penggunaan

- Jumlah pemain (2 - 4), petak papan (40), dan kartu dibaca dari file konfigurasi yang di mana tidak bisa menggunakan array statis
- Pengelolaan memori otomatis: (std::unique_ptr): Digunakan dalam `std::vector<std::unique_ptr<Petak>>`. Karena Petak bersifat polimorfik (ada Street, Railroad, Utility), kita harus menggunakan pointer. STL `unique_ptr` memastikan bahwa saat vector dihancurkan atau petak dihapus, memori akan otomatis dibersihkan tanpa risiko memory leak (tidak perlu delete manual).
- Efisiensi pencarian dan Iterasi: Hampir seluruh entitas game (Pemain, Papan, Log) disimpan dalam std::vector karena memberikan akses cepat (O(1)) melalui indeks dan sangat efisien saat melakukan iterasi untuk memproses giliran pemain.
- Manipulasi String yang Dinamis (`std::string`): Digunakan untuk menangani nama pemain, deskripsi kartu, dan log transaksi. std::string jauh lebih aman dan fleksibel dibandingkan char array tradisional karena ukurannya bisa bertambah otomatis sesuai kebutuhan.

```
std::sort(activePlayers.begin(), activePlayers.end(), [](Player *a, Player *b) {
    if (a->getMoney() != b->getMoney())
        return a->getMoney() > b->getMoney();
    return a->getOwnedProperties().size() > b->getOwnedProperties().size();
});
```

4.5.2. Penjelasan & Alasan Penggunaan

Pengurutan Pemenang `(std::sort)` Digunakan pada `GameManager::determineMaxTurnWinners()` untuk menentukan peringkat pemain berdasarkan saldo uang, jumlah properti, dan jumlah kartu secara otomatis.

```
std::shuffle(specialDeck.begin(), specialDeck.end(), rng);
```

4.5.3. Penjelasan & Alasan Penggunaan

Std::shuffle digunakan untuk mengacak tumpukan kartu kesempatan/dana umum agar permainan tidak dapat diprediksi

4.6. Konsep OOP lain

Sesuai dengan kriteria nilai bonus, proyek kami juga menerapkan konsep *Composition*. *Composition* adalah salah satu bentuk relasi kelas ("has-a") yang erat, di mana siklus hidup (*lifecycle*) dari objek komponen/bagian terikat sepenuhnya pada objek pemilik/utamanya. Jika objek pemilik dihancurkan, maka objek bagiannya juga ikut hancur. Berikut adalah cuplikan kode deklarasi kelas *GameManager* pada file *include/core/GameManager.hpp* yang menunjukkan relasi komposisi dengan kelas *Dice*:

```
class GameManager {
private:
  std::vector<Player> players;
  std::vector<std::unique_ptr<Petak>> board;
  Dice dice; // <--- Implementasi Composition
  CardDeck chanceDeck;
  CardDeck chestDeck;

  // ... deklarasi metode dan atribut lainnya
};
```

4.6.1 Penjelasan & Alasan Penggunaan

Pada potongan kode di atas, variabel *dice* diinstansiasi langsung sebagai *value type* member dari kelas *GameManager* (bukan sebagai pointer referensi yang dititipkan dari luar). Konsep ini merupakan contoh penerapan *Composition* yang valid karena:

- *Relasi "Memiliki" yang Kuat:*
  GameManager (sebagai pengelola permainan utama) memiliki atribut perangkat lempar dadu (*Dice*) yang menjadi satu kesatuan mesin permainan.
- *Keterikatan Siklus Hidup (Lifecycle)*
  Karena *dice* dideklarasikan secara langsung di memori stack milik kelas *GameManager*, maka saat instance *GameManager* itu dibuat, *dice* akan ikut terbuat. Dan yang terpenting, ketika permainan selesai dan instance *GameManager* dihancurkan dari memori, objek *dice* juga dipastikan ikut musnah. *Dice* tersebut tidak memiliki identitas atau eksistensi independen di luar lifecycle dari *GameManager*.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. COM

Agar permainan Nimonspoli dapat dimainkan secara single-player, kami mengimplementasikan sistem *Computer Opponent* (COM) melalui kelas *ComPlayerAI* yang terletak pada src/core/com/ComPlayerAI.cpp. Pendekatan utama kami adalah memisahkan logika pengambilan keputusan (*decision-making*) otonom dari interaksi user standar. AI yang kami bangun tidak hanya sekadar mengambil keputusan acak, melainkan menggunakan kalkulasi heuristik dan probabilitas berbasis kondisi finansial pemain bot tersebut. Berikut adalah dua implementasi logika utama pada bot kami:

- Logika Mengikuti Lelang (*Auction Bid*)
  Bot memiliki algoritma penawaran lelang yang konservatif dan dinamis. Bot menetapkan batas maksimal harga beli (*maxWillingToPay*) sebesar 80% dari harga asli properti untuk menghindari pemborosan kas. Bot juga selalu mengecek ketersediaan uang sebelum menawar. Jika batas penawaran masih aman, bot akan menaikkan harga lelang dengan rasio peningkatan minimum 50 atau 10% dari harga properti. Untuk membuat bot tidak terlalu mudah diprediksi oleh pemain manusia, kami menambahkan faktor probabilitas sebesar 60% (*decide*(0.60)) di mana bot bisa saja tiba-tiba mundur dari lelang meskipun masih memiliki cukup dana.
- Logika Likuidasi (*Liquidation Option*)
  Dalam kondisi terdesak di mana bot harus melikuidasi aset, bot menggunakan algoritma sederhana yang secara otomatis (rekursif/langsung) memilih opsi pertama yang tersedia (return 1) untuk mempercepat resolusi kebangkrutan tanpa menghentikan game loop.

Cuplikan Kode dari file src/core/com/ComPlayerAI.cpp :

```
int ComPlayerAI::decideAuctionBid(const Player &p, const PetakProperti *prop,
                                  int currentHighestBid) {
  int propertyValue = prop->getHargaBeli();
  int maxWillingToPay = static_cast<int>(propertyValue * 0.8);
  int money = p.getMoney();

  int maxBid = std::min(maxWillingToPay, money);
```

```
  if (currentHighestBid >= maxBid)
    return 0;

  int bid = currentHighestBid + std::max(50, propertyValue / 10);
  if (bid > maxBid)
    return 0;

  // Faktor probabilitas 60% agar AI tidak selalu kaku
  if (!decide(0.60))
    return 0;

  return bid;
}

int ComPlayerAI::chooseLiquidationOption(const Player & , int numOptions) {
  if (numOptions > 0)
    return 1;
  return 0;
}
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
MODUL CETAK_PAPAN	13524145, 13524127	13524145, 13524127
MODUL LEMPAR_DADU	13524145	13524145
MODUL ATUR_DADU X Y	13524145	13524145
MODUL CETAK_AKTA	13524145	13524145
MODUL CETAK_PROPERTI	13524132	13524132
MODUL BELI (OTOMATIS)	13524145	13524145
MODUL BAYAR_SEWA (OTOMATIS)	13524132	13524132
MODUL BAYAR_PAJAK (OTOMATIS)	13524132	13524132
MODUL GADAI	13524143	13524143
MODUL TEBUS	13524143	13524143
MODUL BANGUN	13524113	13524113
MODUL LELANG (OTOMATIS)	13524143	13524143
MODUL FESTIVAL (OTOMATIS)	13524113	13524113
MODUL BANGKRUT DAN AMBIL ALIH ASET (OTOMATIS)	13524127	13524127
MODUL SIMPAN	13524143	13524143
MODUL MUAT	13524113	13524113

MODUL CETAK_LOG	13524113	13524113
MODUL MENANG (OTOMATIS)	13524132	13524132
MODUL KESEMPATAN DAN DANA UMUM (OTOMATIS)	13524127	13524127
MODUL GUNAKAN KEMAMPUAN	13524127	13524127
MODUL DROP KARTU KEMAMPUAN (OTOMATIS)	13524113	13524113

7. Lampiran

Lampiran A. Form Asistensi

**Pranala Form Gdrive Asistensi Kelompok Kami: https://drive.google.com/file/d/1PGk_jCd_9fxoD3PHj5jGWkhtsIzpt3qL/view?usp=sharing*

Lampiran B. Repository Kelompok

**Pranala Repository Kelompok Kami: https://github.com/Labpro-22/tugas-besar-1-psi*